CS2212 Group Project

Testing Documentation

Version	Date	Author(s)	Summary of Changes
1.0	2025.12.01	Peiyong Wang, Zhixian Wang, Xinyan Cai, Junqi Zheng, Jialin Li	Create introduction, test plan including unit, integration, validation and system test, and summary sections.

Table of Content

Section	Discription
1. Introduction	Provides an overview of the Adventure Game Engine, including the testing objectives, scope, and references.
2. Test Plan	Details the comprehensive testing strategy covering Unit, Integration, Validation, and System testing levels with specific test cases.
3. Summary	Concludes the document by outlining the testing methodology, tools used, and key project deliverables.

Comments

1. Introduction

1.1 Overview

The software system developed by Group02 is a Java-based **Adventure Game Engine** designed to play point-and-click style adventure games. The application loads game data (locations, objects, characters, and rules) from an external JSON/XML file, allowing users to explore a virtual world through a graphical user interface.

Key features of the system include:

- **Command Processing:** Parsing and executing user actions such as `Go`, `Pick Up`, `Use`, and `Talk`.
- **Game State Management:** Tracking user inventory, current location, and world state changes based on rule interactions.
- **Interactive UI:** Displaying location images, descriptions, and providing visual feedback for user commands.
- **Data-Driven Design:** Decoupling the game engine from the story content to allow support for multiple game scenarios.

1.2 Objectives

The primary objective of this project is to apply software engineering principles to design, implement, and verify a robust adventure game engine.

Specific objectives for this Testing Documentation include:

- To ensure the implementation meets all functional requirements defined in the **Project Specification**, including navigation, inventory management, and complex rule matching (Give/Use).
- To expose and mitigate defects in logic, parsing, state transitions, and boundary conditions.
- To validate non-functional requirements such as system stability and data persistence (Save/Load functionality).

1.3 References

1. CS2212A Group Project Specification - https://westernu.brightspace.com/content/enforced/127086-UGRD_1259_3953/CS2212A%20Project%20Specification.html
2. CS2212 Group Project Requirements Documentation (Group 2) -Gitlab, Requirements Documentation
3. CS2212 Group Project Design Documentation (Group 2) -Gitlab, Design Documentation

Comments

2. Test Plan

- **Purpose**
  - Provide a detailed plan for verifying and validating the implemented Adventure Game.
  - The plan explicitly covers Unit, Integration, Validation and System testing as required by the assignment.
- **Scope**
  - The plan addresses the game's core modules: command parsing and execution, game state model, rules engine (GiveRule/UseRule), inventory, locations and characters, data loader/saver, and UI hooks (testable via non-graphical substitutes).
  - Non-functional tests (basic performance and stability) are included in System testing.
- **Objectives**
  - Ensure the implementation meets the functional requirements defined in the project specification.
  - Expose and mitigate defects in logic, parsing, state transitions, persistence and boundary conditions.
  - Provide traceability from requirements to test cases and acceptance criteria.

Comments

2.1 Unit Testing

Goal: Verify the correctness of individual classes and methods in isolation.

Method: Write JUnit 5 tests for public methods and for internal behaviors where necessary (for critical logic). Use test doubles/mocks for external dependencies.

Entry Criteria: Source code compiled, classes instantiable; developers have added unit tests to the test/ directory.

Exit Criteria: All defined unit-level tests executed locally; targeted unit coverage metric achieved for critical classes (e.g., 80% lines for Rules/Commands/GameEngine) OR all stated unit test cases implemented.

Target Modules and Representative Test Cases (with rationale)

A. Data Model and Collections

Module	Test Cases	Rationale
GameObjectCollection	add, remove, getByName, **case-insensitive name lookup**, iteration.	Covers boundary conditions for object stores and core name matching.
Inventory	addObject success and failure when capacity reached; removeObject; containsObject; getAllObjects size changes.	Covers capacity logic and invariants.
Location	connections list integrity, add/remove objects/characters, connection lookups by name, unreachable/isolated location edge cases. **Dynamic addConnection/removeConnection** state validation.	Covers spatial model, path invariants, and support for **dynamic connection creation**.
GameEngine/GameData	Test **turnCount** increment logic and verification of **turnLimit** being reached leading to an EndGame state.	**Turn Counting** is a mandatory functional requirement.

B. Rules and Matching (Refined for GiveRule Restriction)

Module	Test Cases	Rationale
GiveRule.applicable(...)	Test only with **exact name matches**; case variations, leading/trailing whitespace. **Critical Test:** Verify that attempting to match by attribute **always fails**.	**GiveRule is strictly limited to name matching**. This ensures logic adheres to the specific implementation constraint.
UseRule.applicable(...)	Test with **Attribute matches** ("Hard" attribute + "Window"); **Exact name matches**; **Mixed matches** (Name + Attribute); case variations.	Ensures the implemented **Attribute-based matching** works correctly, as this complexity is confined to UseRule.
UseRule	Test Rule execution when it carries the **requestEnd** flag, verifying it signals the GameEngine to perform end-game handling.	Validates the mechanism for **ending the game via a specific Use command**.

C. Individual Command Classes (Focus on Feature Variations)

Command	Test Cases	Rationale
ExamineCommand	**Unit:** Returns description; handles unknown names; when an object **contains items** (e.g., Mysterious Box), verifies the contained objects are correctly **added to the Location's object list**.	Validates the logic for **discovering contents of container objects**.
UseCommand	**Unit:** validate correctly matches UseRule based on name and **attributes**; execute verifies used objects are removed, resultingObjects are added, and **dynamic Connections** are created/updated.	Ensures correctness of **Attribute-based rule application** and **dynamic state transitions** (e.g., new connections).
GoCommand	**Unit:** Validates connection existence (including **newly created connections**); verifies that moving to an **EndLocation** correctly triggers an EndGame state.	Covers dynamic connection usage and the logic for the **EndLocation ending condition**.

PickCommand	**Unit:** Validate presence and pickable flag; removing from location and adding to inventory; error when not present. **Integration:** pick then use/give sequence relies on picked items.	Ensures core inventory addition logic.
DropCommand	**Unit:** Validate removal from inventory and addition to location; error when item not in inventory. **Integration:** drop then pick back to verify persistence of state.	Ensures core inventory removal logic.
TalkCommand	**Unit:** Validate character presence; returns dialogue text and optional side-effects.	Ensures dialogue sequence logic.
GiveCommand	**Unit:** Validate character present and inventory contains object; matchedRule selection (only by name); **requestEnd for end-game rules**. **Integration:** complete give sequences that may produce resulting objects or trigger endings.	Ensures complex rule-based interaction logic.

D. Data Loader & Persistence

Module	Test Cases	Rationale
DataLoader.loadGameData(file)	valid file → returns fully populated GameData; invalid/malformed file → throws documented exception; checks **referential integrity** post-load.	Covers robustness of parsing and prevents runtime errors.
Persistence (Save/Load)	**Unit:** Test saveState() and loadState() for GameEngine/GameData, verifying that key fields (currentLocation, turnCount, inventory contents) remain consistent across serialization/deserialization.	Validates the correctness of the **Add-on feature: Persistence (Save/Load)**.

Comments

2.2 Integration Testing

Goal: Verify correct interactions between components (commands, engine, data model, UI hooks, persistence).

Approach: Use integration JUnit tests that wire real instances together but substitute GUI components with SilentGameUI and images with TestImageLoader. Focus on sequences and dataflow rather than isolated method behavior.

Entry Criteria: Unit tests completed for related units; integration test scaffolding (SilentGameUI) available.

Exit Criteria: All integration scenarios implemented as test cases and executable in local environment.

Integration Scenarios (Detailed Test Cases)

Case ID	Scenario Description	Steps and Verification Focus	Rationale
I1	**Command parsing to execution (Give command path)**	Invoke engine.processCommand("give O to C"). **Observe:** Inventory item removed; resulting objects added; if rule.endGame → engine performs end-of-game handling; **Turn Count incremented**.	Exercises parser, command factory, command validate/execute, rules lookup, and engine post-execution logic.
I2	**Use Command Path (Attribute-based match & Dynamic Connection)**	1. Use **Metal Rod** (Attribute: Hard) on **Study Window**. 2. **Verification:** Study Window is replaced by the **WindowEscape** connection; the correct message is displayed.	Validates the **Rule Engine correctly handles attribute matching** (via UseRule) and **dynamic Connection creation**.
I3	**DataLoader → GameData consistency**	Load via DataLoader, check referential integrity (Rules vs. Objects), and the presence of startLocation and **EndLocation(s)**.	Prevents runtime NPEs and verifies loaded data structure consistency.
I4	**Multi-command sequences (Container, Give, Use)**	Execute the sequence: PICKUP **Cup** → USE **Cup + Faucet** → GIVE **Cup of Water** to **Ghostly Boy** → USE **Child's Emblem + Old Cabinet**. **Verification:** Assert intermediate states (Cup of Water, Child's Emblem, Glowing Lantern) are correctly gained/removed.	Verifies the flow of **container item usage**, **Give rule activation** (by name), and **multi-step puzzles**.
I5 (New)	**Persistence Cycle Test**	1. Execute I4 steps up to acquiring the **Child's Emblem**. 2. engine.saveGame(). 3. engine.loadGame(). 4. Continue executing the remaining steps of I4. **Verification:** Game state (Inventory, Location, Turn Count) is restored perfectly, and subsequent commands execute without error.	Validates the robustness of the **Save/Reload feature** within an active gameplay session.

Comments

2.3 Validaion Testing

Goal: Validate that implemented features satisfy the requirements (requirement-level verification). This is black-box testing mapped to requirements.

Method: For each functional requirement, define at least one test case that demonstrates the expected behavior. Each validation test case must reference the requirement ID and provide a pass/fail oracle.

Entry Criteria: Requirements list finalized; integration tests prepared.

Exit Criteria: Requirement → test mapping exists and tests are automatable.

Representative Mapping and Test Design (Examples)

REQ ID	Test Design	Justification/Expected Behavior (Oracle)
REQ R1/R3	**Navigation and EndGame (Side Path)**: Use **Attribute-based USE** (Metal Rod on Study Window) to create **WindowEscape**. Then, use GO: WindowEscape to move to **GardenExit**.	**Oracle:** New connection is created; entering **GardenExit** (an EndLocation) causes the GameEngine to **immediately trigger EndGame**.
REQ R1/R3	**Rule-based EndGame (Main Path)**: Execute the final required action: USE: Glowing Lantern in Secret Chamber.	**Oracle:** The Use Rule triggers the state change and the GameEngine correctly triggers EndGame based on the **UseRule's requestEnd flag**.
REQ R2	**Item Pickup and Inventory/Container**	EXAMINE **Mysterious Box**; confirm **Secret Key** is added to the location list. Then PICKUP **Secret Key**; confirm Inventory contains it.
REQ R4	**Persistence (Save/Load)**	Save a GameData instance to file after key state changes (e.g., inventory contains **Glowing Lantern**). Load it back and attempt to continue the game flow.
REQ R5 (Turn Count)	**Turn Limit Ending**	Set a small turnLimit; execute commands until the limit is reached.

Comments

2.4 System Testing

Goal: Verify the system as a whole in an environment approximating user deployment and validate non-functional requirements (performance and stability basics).

Entry Criteria: Integration tests complete; baseline build and test harness available.

Exit Criteria: All system test scripts prepared and runnable; non-functional thresholds defined and documented.

System Test Cases (Detailed)

Case ID	Scenario Description	Steps and Verification Focus
S1	**End-to-End scenario execution (Main Path - Happy Path)**	Steps: Load **DataFile.json**, simulate the defined **Main Path (SecretChamber Ending)** [1] by invoking the sequence of commands. Focus: end-to-end state transitions, interaction of all components, display hooks invoked, and successful **EndGame trigger via UseRule**.
S2	**Alternative Endings and Branch Coverage**	Steps: Execute command sequences that exercise all defined alternative endings: 1. **Side Path (EndLocation)** ending. 2. **Turn Limit** ending. Focus: Ensure all ending paths are reachable and trigger the correct end-game state/message.
S3	**Persistence and Recovery Stability**	Steps: During a complex scenario, call **save**, simulate an application restart, **load** the game, and continue the scenario to a successful conclusion. Focus: Data consistency and overall system stability following data recovery.
S4	**Non-Functional - UI/Usability Check**	Steps: Execute a standard sequence of commands and check the UI. Focus: Verify all required UI elements (Location Name/Picture, Items, Connections, Inventory, Turn Count) are always visible, correctly updated, and responsive throughout the session.

Comments

3. Summary

This Test Plan outlines a comprehensive mixed-method strategy (white-box and black-box) designed to verify the Adventure Game's compliance with functional requirements and robustness against defects . The methodology is structured around four distinct levels: Unit testing to isolate logic correctness using JUnit 5, Integration testing to verify component interactions via a `SilentGameUI`, Validation testing to map specific checks directly to requirements (such as navigation and inventory), and System testing to validate end-to-end storylines and data persistence .

To ensure execution reliability, the team will utilize JDK 17/22 and a command-line test runner, placing the highest priority on validating command parsing and rule matching . The key deliverables include this plan, the JUnit source code, system scripts, and a final Test Report, with primary risks regarding GUI dependencies mitigated through the use of test doubles and image loader substitutes .

Comments